UniBus Live: A Real-Time Bilingual University Bus Tracking System with Reliable Dual-Source GPS Acquisition

Md. Shishir Hossain, Md. Sajib Hasan, Md. Showkat Hossen
Department of Computer Science and Engineering
NPI University of Bangladesh
Manikganj, Dhaka, Bangladesh
mdshishirhossain3@gmail.com

***Abstract*—Students at many universities wait at bus stops with no reliable information about where their bus is or when it will arrive, leading to wasted time and uncertainty. This paper presents UniBus Live, an end-to-end real-time bus tracking system for university transport comprising a passenger mobile application, a web administration console, and a central application server backed by a 31-model relational schema. To remain reliable in low-cost deployments, the system acquires vehicle positions from two independent sources—a fixed vehicle-grade GPS tracker as the autonomous, driver-independent primary feed and the driver's smartphone as an optional automatic fallback—reconciled by a health-based arbitration component. Arrival times are estimated geometrically by projecting the live position onto the route polyline and dividing the remaining along-path distance by a rolling-average speed, with each estimate carrying a confidence label. The passenger interface is fully bilingual (English and Bangla) and remains informative before departure and during signal loss. We evaluate the system in a seven-week pilot spanning 9 routes, 3 buses, and 71 completed trips, during which 46,548 GPS reports were ingested. The dual-source design was exercised in practice, with 64% of trips tracked by hardware and 36% by the smartphone fallback; median position-acquisition latency was 5.9 s. These results demonstrate the feasibility of reliable, low-cost, real-time bus tracking for university transport.**

***Index Terms*—real-time systems, GPS tracking, public transport, estimated time of arrival, WebSocket, mobile application, intelligent transportation systems**

I. INTRODUCTION

University bus services are a daily necessity for thousands of students, yet at many institutions—particularly in developing regions—they operate without any means for riders to see where a bus is in real time. A student arriving at a stop has no way of knowing whether the bus left five minutes ago or is fifteen minutes away. The resulting uncertainty causes missed buses, long unsheltered waits, and avoidable crowding. Commercial fleet-tracking products exist but are typically costly, designed for logistics rather than passenger-facing use, and rarely localized to the language of the student population.

This paper presents *UniBus Live*, a complete real-time tracking platform built specifically for university transport and deployed on a real campus fleet. The system is designed around three goals that distinguish it from a generic vehicle tracker. First, *reliability*: positions come primarily from a fixed hardware tracker that operates autonomously—no driver action, driver-owned device, or driver data connection is required—while an optional smartphone fallback can take over automatically when a hardware fix is unavailable, so coverage degrades gracefully instead of disappearing. Second, *passenger-centric information*: the system computes per-stop arrival estimates, surfaces the bus state before a trip officially begins, and alerts a rider as their chosen drop-off stop approaches. Third, *localization*: the entire passenger experience is bilingual (English and Bangla), reflecting the language of its riders.

The contributions of this work are:

- An end-to-end, deployed architecture for real-time university bus tracking centred on an autonomous fixed GPS tracker with an optional, health-arbitrated smartphone fallback, providing reliable coverage at low cost.
- A lightweight geometric ETA estimator that projects the live vehicle position onto the route polyline and derives per-stop arrival times from a rolling-average speed, annotated with a confidence level.
- A passenger-facing design contribution—bilingual UX with pre-trip vehicle awareness, staleness signalling, and proximity-based drop-off alerts—that keeps the interface informative in the edge conditions where naive trackers show a blank map.
- An evaluation of the deployed system over a seven-week pilot using operational data, reporting acquisition latency, source-fallback usage, and ETA estimation behaviour.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the system architecture and Section IV its implementation. Section V presents the pilot evaluation, and Section VI concludes.

II. RELATED WORK

A. Campus and Real-Time Bus Tracking Systems

Several student-facing bus tracking systems have been reported. Sharif *et al.* [1] present a real-time campus bus tracking mobile application that pairs an on-board GPS tracker with an IoT passenger counter to show bus location and occupancy, and Premkumar *et al.* [2] combine GPS tracking with a schedule view and push notifications for a college fleet. Such systems establish the practical value of live campus tracking but typically depend on a single on-board positioning source and expose an English-only interface. UniBus Live differs in two respects relevant to low-cost, localized deployments: it ingests two independent position sources—an autonomous fixed hardware tracker as primary, with the driver's smartphone as an optional fallback—and its passenger experience is fully bilingual.

B. Smartphone-Based and Crowdsourced Positioning

A complementary line of work uses smartphones themselves as the position source. Zhou *et al.* [3] pioneered participatory sensing for bus arrival prediction, inferring bus location from passengers' phone sensor data without dedicated hardware, and

Wepulanon *et al.* [4] propose a real-time arrival system driven entirely by crowdsourced smartphone data. These approaches lower hardware cost but depend on sufficient passenger participation and suffer coverage gaps when few contributors are aboard. UniBus Live inverts this relationship: normal operation depends on no phone at all—the primary source is an autonomous fixed tracker—while the driver's phone can serve as an optional fallback, arbitrated automatically by source health, so phone-based sensing supplements rather than carries the system.

C. Bus Arrival-Time (ETA) Prediction

Arrival-time prediction is a well-studied problem. Reich *et al.* [5] survey ETA prediction methods for public transport and organize them by input data, while Singh and Kumar [6] review artificial-intelligence approaches to bus arrival-time prediction. A broad spectrum of data-driven predictors has since been demonstrated: classical machine learning over big-data frameworks, such as support-vector regression on Spark [7]; fully connected neural networks that predict arrival and departure times across hundreds of routes [8], [9]; transformer encoders that model long-range temporal dependence in bus trajectories [10]; and Google's BusTr [11], which translates real-time traffic forecasts into delay predictions at global scale. These approaches achieve strong accuracy but depend on large historical trajectory datasets and external traffic feeds that a new, single-campus deployment does not yet have. UniBus Live therefore adopts a lightweight geometric estimator that requires no training data, while logging every prediction so a data-driven model can be trained as operational history accumulates.

D. GPS Quality, Filtering, and Map Matching

Because raw GPS is noisy, considerable work addresses reconciling observed fixes with the road network. The Hidden Markov Model map-matching method of Newson and Krumm [12] is a widely adopted, principled approach that accounts for measurement noise and road-network layout. Full map-matching is, however, heavier than necessary when the vehicle is known to follow a small set of fixed, pre-defined routes. UniBus Live instead applies a pragmatic subset—accuracy gating, a physical-plausibility (teleport) guard, and projection of the fix onto the known route polyline—sufficient to stabilize the displayed position and drive along-route ETA estimation.

E. Open Telematics Platforms

Open-source telematics servers such as Traccar [13] provide device-protocol handling for a wide range of trackers together with a REST API and geofencing primitives. UniBus Live uses such a platform for raw device ingestion and builds a passenger-facing, ETA-aware, bilingual application layer on top of it, rather than reimplementing low-level device communication. In contrast to single-source prototypes, simulation-only studies, and data-hungry prediction models, UniBus Live contributes a *deployed*, low-cost system emphasizing source reliability, a training-data-free ETA estimator, and a localized, edge-condition-aware passenger experience, evaluated on a real campus fleet.

III. SYSTEM ARCHITECTURE

UniBus Live follows a three-tier, event-driven architecture comprising four cooperating components: (i) a passenger mobile application, (ii) a web-based administration console, (iii) a central application server, and (iv) a vehicle GPS-acquisition layer. The components communicate over a combination of stateless REST calls for command/query operations and a persistent WebSocket channel for low-latency push of live vehicle state. Fig. 1 presents the complete system architecture, and Fig. 2 traces the real-time flow of a position report from the bus to the passenger's screen.

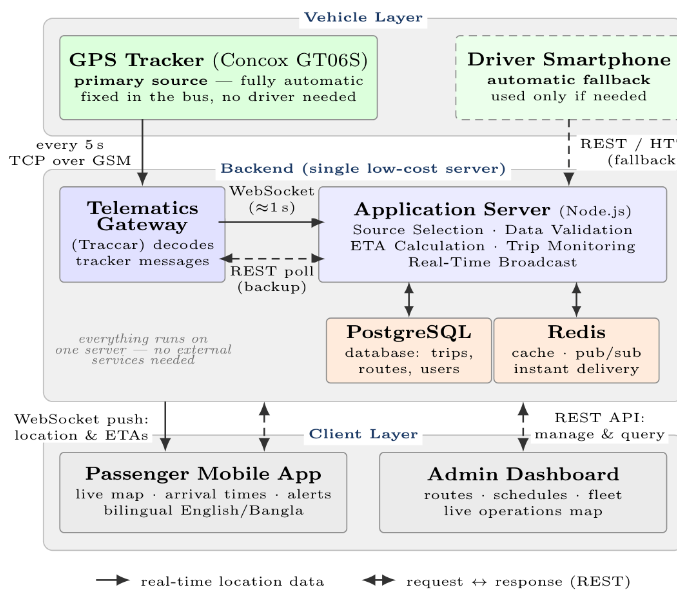


Fig. 1. UniBus Live system architecture. The GPS tracker is the primary, fully automatic source; the driver's smartphone is only a fallback. All backend components share one low-cost cloud server.

Fig. 2. Real-time data flow: the life of one position report, from the GPS fix on the bus to the passenger's screen, grouped into acquisition, processing, and delivery phases. Steps 3–5 are detailed in Sections IV-A–IV-C.

A. Application Server

The server is implemented in TypeScript on Node.js [14] using the Express framework [15]. Persistence is provided by PostgreSQL [17] through the Prisma [16] object–relational mapper; the schema comprises 31 relational models organised into six functional domains (Fig. 3): identity and authentication, fleet and devices, routing and scheduling, trip and tracking, communications and passenger, and audit and operations. An in-memory Redis [18] instance serves two roles: a short-lived cache for hot read paths and the publish/subscribe back-plane for the WebSocket layer, letting the realtime tier scale horizontally without losing fan-out consistency. Realtime delivery uses Socket.IO [19] over the WebSocket protocol [24], with each active trip represented as a logical *room*; a passenger tracking a trip joins that trip's room and thereafter receives only its events, bounding per-client traffic to the single vehicle being observed.

Identity & Auth User, Session, Role, OTP	**Fleet & Devices** Bus, GpsDevice, Assignment
Routing & Schedule Route, Stop, RouteStop, Schedule	**Trip & Tracking** Trip, LocationLog, TrackingState
Comms & Passenger Notification, Favorite, Occupancy	**Audit & Ops** AuditLog, TripEvent, IngestLog

Fig. 3. The 31-model relational schema, grouped into six functional domains.

B. GPS-Acquisition Layer

Vehicle positions originate from two independent source types. The primary source is a fixed, vehicle-grade Concox GT06S GPS/GSM tracker hard-wired to the bus—the satellite-positioning principles underlying such receivers are treated comprehensively in [23]—reporting to a self-hosted Traccar telematics server. An optional secondary source is the driver's smartphone, which can publish location through an authenticated REST endpoint when no hardware fix is available; normal operation uses the tracker alone and requires no driver device, action, or connectivity. The application server ingests Traccar data through two complementary paths: a session-cookie WebSocket subscription that *pushes* each new fix within about a second of Traccar receiving it, backed by an 8 s REST polling job as a fallback when the socket is unavailable. It then reconciles the two source types through a source-arbitration component (Section IV-A). Both Traccar and the application server are self-hosted on a single low-cost virtual private server, keeping recurring infrastructure cost within a student-project budget.

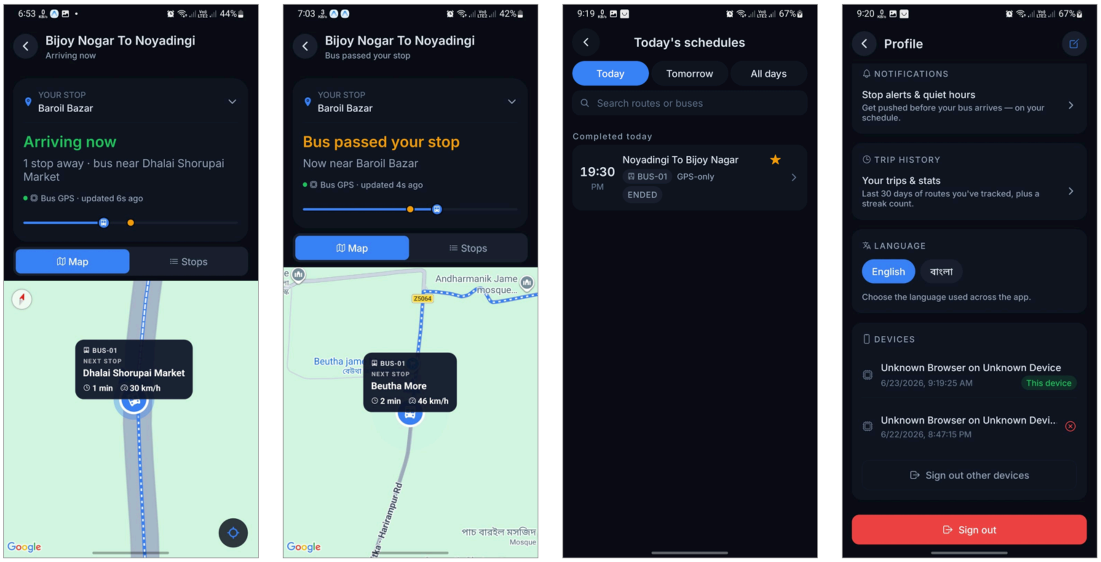
C. Client Applications

The passenger application is built with React Native [20] and Expo [21], rendering live vehicle and route geometry on a map. It uses a foreground location service for continued tracking while backgrounded and push notifications for alerts, and its interface is fully bilingual (English and Bangla); representative passenger screens are shown in Fig. 4. The administration console is a server-rendered React (Next.js) [22] application integrating the Google Maps JavaScript API for the live operations map and the route-builder tool; it also manages buses, GPS devices, schedules, and user roles through the same REST API.

IV. IMPLEMENTATION

A. Location Ingestion and Multi-Source Arbitration

Each inbound fix—polled from Traccar, pushed over its WebSocket, or posted by a driver device—is recorded with both its device-reported timestamp and its server-receipt timestamp, and tagged with its source. For every bus the server maintains one *source-tracking state* per active source and a single *canonical tracking state* representing the position passengers see. When a hardware tracker and a driver phone are both live, an arbitration routine selects the canonical source by health score and a configurable priority rank, preferring a healthy hardware tracker but transparently falling back to the phone if the tracker goes silent. Because arbitration operates per fix, a bus can hand over between sources mid-trip without any passenger-visible artefact: the canonical marker simply continues from the freshest healthy source. Fallback participation is opt-in per trip—the phone enters arbitration only when a driver is assigned to the trip and is actively publishing—so what is automatic is the *selection* among live sources, while trips scheduled without a driver rely on the tracker alone. Every accepted fix is also appended to a per-trip location log, preserving the complete trajectory for later analysis.



(a) Live tracking with ETA (b) On-route: next stop & speed (c) Today's schedules (d) Language: English / Bangla

Fig. 4. The bilingual passenger application. (a) live tracking of a moving bus with the next-stop ETA; (b) on-route status showing the next stop and current speed; (c) today's schedules; (d) the in-app English/Bangla language toggle. Place names render in both scripts.

B. Location-Quality Filtering

A filtering pipeline gates every fix before it can move the canonical position. The principal, operator-configurable rules are: (i) an accuracy gate that soft-rejects fixes worse than 120 m and hard-rejects those worse than 250 m; (ii) a teleport guard that rejects implied inter-fix speeds above 120 km/h; (iii) a minimum-movement threshold of 4 m below which the bus is treated as stationary, suppressing GPS jitter; and (iv) a freeze/release hysteresis that holds the marker still under poor accuracy and resumes motion only on a confident displacement. A short-window rolling average of accepted speeds is maintained per trip and feeds the ETA estimator. The thresholds are deliberately conservative: 120 km/h exceeds any plausible speed on the pilot routes, and the 4 m floor sits above the tracker's typical urban scatter, so legitimate motion is never suppressed.

C. Arrival-Time Estimation

ETAs are computed geometrically against the ordered list of route stops. The route is first reduced to a cumulative along-path distance array. The current fix is projected onto the nearest route segment using a planar (local equirectangular) projection, yielding the bus's progress distance and cross-track offset; this makes the estimate robust to lateral GPS error and to the bus being slightly off the idealised polyline. The remaining distance to each upcoming stop is the difference between that stop's cumulative distance and the bus's progress, divided by an effective speed—the per-trip rolling-average speed when reliable ($\geq$ 5 km/h), otherwise a configurable default of 20 km/h—to yield a per-stop ETA, so a rider receives an estimate for their own chosen stop rather than only the next one. Each estimate carries a HIGH/MEDIUM/LOW confidence label derived from the speed-signal quality. A stop is marked reached when the bus enters an 80 m arrival radius, triggering a stop-arrival event and, for subscribed riders, a push notification. The procedure is summarised in Algorithm 1.

Algorithm 1 **Estimating the ETA to every upcoming stop**

Require: current bus GPS fix; ordered route stops; recent average speed
Ensure: an ETA and a confidence label for each stop ahead

Compute each stop's distance from the route start (cumulative)
Project the bus fix onto the nearest route segment to find how far along the route it has travelled (its *progress*)
if recent average speed $\geq$ 5 km/h **then**
    speed $\leftarrow$ recent average speed
else
    speed $\leftarrow$ 20 km/h ▷ *fallback default*
end if
for all stops ahead of the bus **do**
    remaining $\leftarrow$ (stop's distance) − (progress)
    ETA $\leftarrow$ *remaining* / *speed*
end for
Assign a confidence (High/Medium/Low) from the speed quality
if the nearest stop is within 80 m **then**
    mark it reached and notify subscribed riders
end if
return the ETA and confidence for every stop ahead

D. Trip Lifecycle and Pre-Trip Awareness

A trip transitions through PLANNED PRE_TRIP RUNNING ENDED. A scheduled job opens a pre-trip window before the scheduled departure, during which the rider sees the bus's pre-departure state (parked, approaching origin, or arrived at origin) rather than a blank map; the vehicle auto-promotes to RUNNING on dwelling inside the origin geofence. A staleness job runs every 30 s and flags trips whose last fix has aged beyond a threshold, so a frozen marker is shown as stale rather than live.

E. Realtime Delivery and Offline Behaviour

Accepted canonical updates and recomputed ETAs are published to the relevant per-trip room and, through the Redis adapter, to peer server processes holding subscribers for that trip; authentication is enforced on socket connection and room join. The client retains the most recent state so a transient disconnection degrades to a "last known" view with an explicit staleness indicator, and the passenger app renders the bus at the route origin during the pre-trip phase so the live map is never blank while a rider waits.

F. Device-Side Reporting Configuration

The server-side throttling described in Section V-F reduces redundant *ingestion*, but the tracker still transmitted a fix every few seconds while parked, draining the bus battery and consuming cellular data at the device itself. Because the Concox GT06S is configured over SMS, we set an asymmetric reporting cadence directly on the unit: TIMER,5,300# makes the tracker report every 5 s while moving but only every 300 s while static; PARAM# reads back the active configuration and STATUS# the battery and GSM/GPS state, each acknowledged by return SMS. This cadence is the transmission-layer counterpart to the server-side gate (Table II). We verified the change from two independent vantage points—the per-position timestamps in the Traccar report view and the server's own ingest log—confirming parked-state fixes spaced by minutes rather than seconds, while a live test drive still produced dense five-second fixes. A low-frequency heartbeat keeps the device visible online between reports; wiring the ignition (ACC) line for engine-state-triggered sleep is identified as future work.

V. EVALUATION

A. Deployment and Dataset

UniBus Live was evaluated in a seven-week pilot (6 May–23 June 2026) on an operational university fleet. Table I summarises the dataset. The deployment covered 9 routes and 3 buses (one equipped with a fixed hardware tracker), served 38 registered passenger accounts, and recorded 71 completed trips during which 46,548 raw GPS reports were ingested. Of these, only 864 were retained as in-trip fixes; the remaining ≈98% were redundant reports transmitted while buses were parked—an inefficiency we quantify and address in Section V (Table II). We report results from operational data; figures are representative of a small pilot rather than a city-scale deployment.

TABLE I
PILOT DEPLOYMENT SUMMARY

Quantity	**Value**
Deployment span	7 weeks
Routes	9
Buses	3
Registered passengers	38
Completed trips	71
Raw GPS reports ingested	46,548
In-trip fixes retained	864
Mean GPS fixes per trip	37.6

B. Methodology

All metrics are computed from the production database. Position-acquisition latency is the difference between each fix's device timestamp and its server-receipt timestamp. Source-fallback usage is taken from the canonical source selected per trip. ETA accuracy is measured by logging every predicted arrival instant and later comparing it against the corresponding recorded stop arrival; we report the mean absolute error.

C. Results

1) Position-acquisition latency: Across 46,286 fixes, the median latency from device fix to server receipt was 5.9 s (mean 7.4 s; 90th percentile 9.9 s; 95th percentile 11.1 s). This is bounded primarily by the 8 s Traccar polling interval rather than by network transport. A further median 1.2 s elapsed before a fix was persisted.

2) Source reliability: The dual-source design was exercised in practice rather than remaining a theoretical safeguard: 64% of trips (45 of 70 with a recorded source) were tracked via the fixed hardware tracker and 36% (25) via the smartphone fallback. This confirms that the fallback path carried a substantial share of real trips, sustaining coverage when hardware fixes were unavailable—largely reflecting trips by the two buses not yet fitted with trackers. The design remains hardware-first: the tracker operates with no driver involvement, and as trackers are fitted fleet-wide the smartphone path reduces to a rarely needed, optional safety net.

3) ETA estimation: The geometric estimator produces a per-stop arrival prediction on every accepted fix, each carrying a HIGH/MEDIUM/LOW confidence label. To support quantitative accuracy assessment, the system logs every predicted arrival instant so that the mean absolute error against recorded stop arrivals can be measured as operational data accumulates over continued deployment.

4) Passenger engagement: Of 45 arrival/service notifications delivered during the pilot, 36 (80%) were read, indicating that the passenger-facing alerts were attended to.

D. Limitations

The pilot is small in scale and duration, a subset of trips were operational tests, and one bus carried the hardware tracker; results should therefore be read as a feasibility demonstration. The hardware tracker does not report a per-fix accuracy value, so quality filtering is evaluated at the trip level rather than per fix. Schedule-adherence analysis was limited by incomplete timetable association during the pilot.

E. Discussion: Toward Adaptive Location Reporting

A notable observation from the pilot is that of the 46,548 raw device reports, only 864 were retained as in-trip fixes: the fixed-interval tracker transmitted continuously even while buses were parked, so roughly 98% of reports fell outside any active trip. For a *scheduled* fleet this wastes cellular data and, for a tracker wired to the vehicle battery, stored charge. A more efficient design couples reporting to vehicle state and the timetable: ignition- and motion-triggered reporting (dense while moving, a sparse heartbeat while parked), distance-based thresholds, and server-side gating of heavy processing to scheduled trip windows—which the system's existing pre-trip window already approximates. Motivated by this observation, we implemented both halves of this strategy: server-side trip-aware throttling that restricts full-rate polling to buses in an active trip or pre-trip window, dropping parked buses to a slow heartbeat while leaving on-trip tracking and the WebSocket push path unaffected; and device-side reconfiguration to 5 s moving / 300 s parked (versus a uniform 5 s previously; Section IV-F), curtailing transmissions, battery drain, and cellular-data use at the source. Finer-grained sleep and distance-based reporting are left as future work. As summarised in Table II, these changes cut parked-state reporting by roughly 60× at the device and idle polling by 7.5× at the server, directly targeting the 98% of reports that the pilot showed were redundant. After the optimization the server ingests essentially only the dense fixes produced during active trips plus a sparse five-minute parked heartbeat per bus, rather than the previous continuous five-second stream.

TABLE II
IDLE-REPORTING EFFICIENCY OPTIMIZATION

Layer	Parameter	Before	After
Device (GT06S)	Parked report interval	5 s	300 s
Server	Idle poll interval	8 s	60 s

VI. CONCLUSION AND FUTURE WORK

We presented UniBus Live, a deployed real-time university bus tracking system distinguished by reliable dual-source GPS acquisition, a lightweight geometric ETA estimator, and a bilingual, edge-condition-aware passenger experience. A seven-week pilot on a real fleet demonstrated feasibility: the system ingested 46,548 GPS reports across 71 trips with a median acquisition latency of 5.9 s, and its optional smartphone-fallback path carried 36% of trips (largely buses awaiting tracker installation), validating graceful degradation while normal operation requires no driver device. The pilot also exposed that roughly 98% of device reports were redundant idle-state transmissions; we eliminated this waste through a two-sided optimization—server-side trip-aware throttling and device-side interval reconfiguration—cutting idle-period reporting by up to 60× without affecting on-trip tracking. Future work includes learning-based ETA prediction using the accumulating trajectory history, full timetable integration for schedule-adherence analytics, crowd-sourced occupancy at larger scale, and a wider, longer deployment with a formal passenger usability study.

ACKNOWLEDGMENT

The authors thank Mahbubur Rahman Chowdhury for supervision, and the Department of Computer Science and Engineering, NPI University of Bangladesh.

REFERENCES

S. A. Sharif, M. S. A. Suhaimi, N. N. Jamal, I. K. Riadz, I. F. Amran, and D. N. A. Jawawi, "Real-time campus university bus tracking mobile application," in *Proc. 7th ICT Int. Student Project Conf. (ICT-ISPC)*, 2018.

K. Premkumar, K. Pavithra, J. Presiela, D. Priyadarshini, and P. Priyanga, "College bus tracking and notification system," in *Proc. Int. Conf. System, Computation, Automation and Networking (ICSCAN)*, 2020.

P. Zhou, Y. Zheng, Z. Li, M. Li, and G. Shen, "How long to wait? Predicting bus arrival time with mobile phone based participatory sensing," in *Proc. 10th ACM Int. Conf. Mobile Systems, Applications, and Services (MobiSys)*, 2012.

P. Wepulanon, A. Sumalee, and W. H. K. Lam, "A real-time bus arrival time information system using crowdsourced smartphone data: a novel framework and simulation experiments," *Transportmetrica B: Transport Dynamics*, vol. 6, no. 1, 2018.

T. Reich, M. Budka, D. Robbins, and D. Hulbert, "Survey of ETA prediction methods in public transport networks," *arXiv preprint arXiv:1904.05037*, 2019.

N. Singh and K. Kumar, "A review of bus arrival time prediction using artificial intelligence," *WIREs Data Mining and Knowledge Discovery*, vol. 12, no. 4, p. e1457, 2022.

L. Ye and P. Thiengburanathum, "A real-time bus arrival time prediction system based on Spark framework and machine learning approaches: a case study in Chiang Mai," in *Proc. Joint Int. Conf. Digital Arts, Media and Technology (ECTI DAMT & NCON)*, 2020.

N. Rashvand, S. S. Hosseini, M. Azarbayjani, and H. Tabkhi, "Real-time bus arrival prediction: a deep learning approach for enhanced urban mobility," *arXiv preprint arXiv:2303.15495*, 2023.

N. Rashvand, M. Azarbayjani, S. S. Hosseini, and H. Tabkhi, "Real-time bus departure prediction using neural networks for smart IoT public bus transit," *IoT*, vol. 5, no. 4, pp. 650–665, 2024.

S. Jeong, C. Oh, and J. Jeong, "BAT-Transformer: prediction of bus arrival time with transformer encoder for smart public transportation system," *Applied Sciences*, vol. 14, no. 20, art. 9488, 2024.

R. Barnes, S. Buthpitiya, J. Cook, A. Fabrikant, A. Tomkins, and F. Xu, "BusTr: Predicting bus travel times from real-time traffic," in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD)*, 2020, pp. 3243–3251.

P. Newson and J. Krumm, "Hidden Markov map matching through noise and sparseness," in *Proc. 17th ACM SIGSPATIAL Int. Conf. Advances in Geographic Information Systems*, 2009, pp. 336–343.

"Traccar: Open source GPS tracking platform." https://www.traccar.org/

OpenJS Foundation, "Node.js." https://nodejs.org/

OpenJS Foundation, "Express web framework for Node.js." https://expressjs.com/

Prisma Data, Inc., "Prisma ORM." https://www.prisma.io/

PostgreSQL Global Development Group, "PostgreSQL." https://www.postgresql.org/

Redis Ltd., "Redis." https://redis.io/

Socket.IO, "Socket.IO: bidirectional and low-latency communication." https://socket.io/

Meta Platforms, Inc., "React Native." https://reactnative.dev/

Expo, "Expo: universal native apps with React." https://expo.dev/

Vercel, Inc., "Next.js: the React framework." https://nextjs.org/

E. D. Kaplan and C. J. Hegarty, Eds., *Understanding GPS/GNSS: Principles and Applications*, 3rd ed. Norwood, MA, USA: Artech House, 2017.

I. Fette and A. Melnikov, "The WebSocket Protocol," IETF RFC 6455, Dec. 2011.